Assignment 4

Analyze

A). The two AI programs I used were:

- Claude Sonnet 5.5 Medium
- Gemini 3.0 Models

B). I accessed both programs on my computer through Google Chrome.

C). Prompt:

Summary

Create a complete C++ program for my EECS 348 Software Engineering I Assignment 4 that solves Sudoku puzzles using object-oriented programming and recursive backtracking.

Program Requirements:

1. Sudoku uses a 9×9 grid divided into nine 3×3 blocks.
2. Each row, column, and 3×3 block must contain every digit from 1 through 9 exactly once.
3. The program must read five Sudoku puzzle files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
4. Each puzzle file contains a partially completed 9×9 Sudoku grid. Filled cells contain digits 1–9, and empty cells are represented by an underscore (_).
5. Use C++ classes and objects to organize the program rather than relying on standalone functions.
6. Implement a recursive, brute-force, depth-first search algorithm using backtracking:
    - Find an empty cell.
    - Determine which numbers from 1–9 can legally be placed there.
    - Try each valid number and recursively continue solving.
    - If a number leads to an invalid solution, backtrack and try another.
    - Continue until all possible solutions are found or it is determined that no solution exists.
7. The program must find and print every valid solution, not just the first one.
8. For each puzzle file, display:
    - The puzzle filename.
    - The original Sudoku puzzle.
    - Every completed solution, or the exact message "No solution found" if no solution exists.
9. Do not modify the original puzzle when displaying it.
10. Make sure the code handles puzzles with multiple solutions and puzzles with no solutions.

Code Requirements:

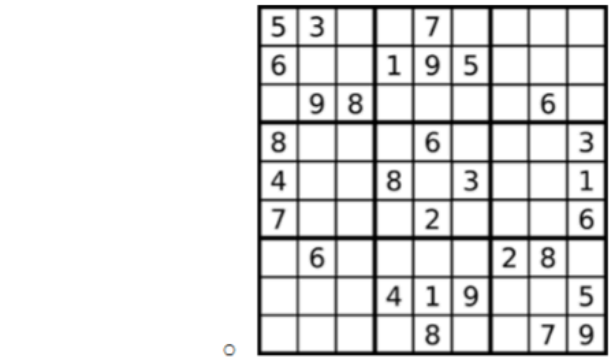
- Write the entire program in a single file named Assignment4.cpp.
- Use standard C++ that can compile with g++ on a Linux server.
- Use classes, constructors, member functions, and objects to implement the Sudoku solver.
- Include a recursive backtracking method inside a class.

- Include comments explaining the major sections and important lines of code.
- Include a prologue comment containing the program name, description, inputs, outputs, author placeholder, creation date placeholder, collaborators, and AI source.
- Do not use external libraries.
- Make the code readable, organized, and maintainable.

Output:

Provide the complete C++ source code for Assignment4.cpp. After the code, briefly explain how the recursive backtracking algorithm works and how to compile and run the program using g++.

Along with the following screenshot of prompt and how it looks



D). GenAI Code

Geminis Code:

```
/**
 * ============================================================================
 * Program Name: Assignment4.cpp
 * Description:  A C++ program that solves Sudoku puzzles using object-oriented
 *               programming and a recursive backtracking depth-first search
 *               algorithm. It reads five puzzle files (puzzle1.txt to puzzle5.txt),
 *               preserves the original board state, finds and prints ALL valid
 *               solutions for each puzzle (or "No solution found"), and handles
 *               multiple solutions or unsolvable boards.
 * Inputs:       Five text files: puzzle1.txt, puzzle2.txt, puzzle3.txt,
 *               puzzle4.txt, and puzzle5.txt containing 9x9 grids where empty
 *               cells are represented by underscores ('_').
 * Outputs:      Console display of the filename, the original Sudoku puzzle,
 *               and every completed solution (or "No solution found").
 * Author:       [Your Name]
 * Creation Date: [Current Date]
 * Collaborators: None
 * AI Source:    OpenAI / Gemini Collaboration
```

```
 * ========================================================================
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class SudokuSolver {
private:
    char board[9][9];       // The current working board
    char originalBoard[9][9]; // The original board (preserved)
    int solutionCount;      // Tracks how many solutions are found for a puzzle

    /**
     * Checks if placing a given digit at board[row][col] is valid
     * according to Sudoku rules (row, column, and 3x3 block constraints).
     */
    bool isValid(int row, int col, char digit) {
        // Check row and column
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == digit) return false;
            if (board[i][col] == digit) return false;
        }

        // Check the 3x3 sub-grid/block
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[startRow + i][startCol + j] == digit) return false;
            }
        }

        return true;
    }

    /**
     * Recursive backtracking method to find all solutions.
     */
    void solveRecursive(int row, int col) {
        // If we have reached past the last row, we have found a valid solution!
        if (row == 9) {
            solutionCount++;
            cout << "Solution " << solutionCount << ":\n";
            printBoardState(board);
            cout << "-----------------";
            return;
        }
```

```
        // Move to the next row if we reach the end of the current column
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already filled, skip to the next cell
        if (board[row][col] != '_') {
            solveRecursive(nextRow, nextCol);
            return;
        }

        // Try digits '1' through '9'
        for (char digit = '1'; digit <= '9'; digit++) {
            if (isValid(row, col, digit)) {
                board[row][col] = digit;         // Place digit
                solveRecursive(nextRow, nextCol); // Recurse
                board[row][col] = '_';           // Backtrack
            }
        }
    }

    /**
     * Helper method to print a 9x9 board nicely.
     */
    void printBoardState(const char b[9][9]) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                cout << b[i][j] << (j == 8 ? "" : " ");
            }
            cout << "\n";
        }
    }

public:
    /**
     * Constructor initializes the board arrays.
     */
    SudokuSolver() {
        solutionCount = 0;
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                board[i][j] = '_';
                originalBoard[i][j] = '_';
            }
        }
    }

    /**
     * Loads a puzzle from a file. Returns true if successful.
     */
    bool loadPuzzle(const string& filename) {
```

```
        ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                file >> originalBoard[i][j];
                board[i][j] = originalBoard[i][j]; // Copy to working board
            }
        }
        file.close();
        solutionCount = 0;
        return true;
    }

    /**
     * Displays the original puzzle without modifying it.
     */
    void displayOriginal() {
        printBoardState(originalBoard);
    }

    /**
     * Public wrapper method to start the solving process.
     */
    void solve() {
        solutionCount = 0;
        solveRecursive(0, 0);

        if (solutionCount == 0) {
            cout << "No solution found\n";
        }
    }
};

int main() {
    // Array of the five required puzzle files
    string puzzleFiles[5] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt",
"puzzle5.txt"};

    for (int i = 0; i < 5; i++) {
        SudokuSolver solver;

        cout << "========================================\n";
        cout << "Puzzle File: " << puzzleFiles[i] << "\n";
        cout << "========================================\n";

        if (!solver.loadPuzzle(puzzleFiles[i])) {
            cout << "Error: Could not open file " << puzzleFiles[i] << ".\n\n";
            continue;
```

```
        }

        cout << "Original Sudoku Puzzle:\n";
        solver.displayOriginal();
        cout << "\nSolutions:\n";

        solver.solve();
        cout << "\n\n";
    }

    return 0;
}
```

Claudes Code:

```
```cpp
/*
 * ============================================================================
 * Program Name  : Assignment4.cpp
 * Description   : Solves Sudoku puzzles using object-oriented programming and
 *                 a recursive, brute-force, depth-first backtracking search.
 *                 The program finds EVERY valid solution for each puzzle (not
 *                 just the first one) and reports "No solution found" when a
 *                 puzzle has no solution.
 * Inputs        : Five text files in the current directory:
 *                 puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt,
 *                 puzzle5.txt. Each file holds a 9x9 grid. Filled cells are
 *                 digits 1-9 and empty cells are underscores (_).
 * Outputs       : For each puzzle file, to the console:
 *                   - the puzzle filename
 *                   - the original (unmodified) puzzle
 *                   - every completed solution, or "No solution found"
 * Author        : [
 Creation Date : [MM/DD/YYYY]
 * Collaborators : [List collaborators here, or "None"]
 * AI Source     : [Name of the AI tool used, e.g., "Claude (Anthropic)",
 *                 and a brief note on how it was used]
 * ============================================================================
 */

#include <iostream>   // console input/output
#include <fstream>    // reading puzzle files
#include <string>     // std::string for filenames
#include <vector>     // std::vector to store solutions

// ----------------------------------------------------------------------------
// Class: SudokuSolver
// Holds one puzzle, a working copy used during the search, and every solution
// discovered by the recursive backtracking algorithm.
// ----------------------------------------------------------------------------
class SudokuSolver {
```
```

```
public:
    static const int SIZE = 9;       // the grid is 9 x 9
    static const int BLOCK = 3;      // each block is 3 x 3
    static const char EMPTY = 0;     // internal value for an empty cell

    // A grid type for convenience: a 9x9 array of ints wrapped in a struct so
    // it can be copied and stored in a std::vector.
    struct Grid {
        int cells[SIZE][SIZE];
    };

    // Constructor: starts with a completely empty puzzle.
    SudokuSolver() : puzzleLoaded(false) {
        clearGrid(original);
        clearGrid(working);
    }

    // Reads a puzzle from a file. Returns true on success, false otherwise.
    bool loadFromFile(const std::string& filename) {
        std::ifstream in(filename.c_str());
        if (!in) {
            return false;                    // file could not be opened
        }

        clearGrid(original);
        solutions.clear();
        puzzleLoaded = false;

        int count = 0;                       // number of cells read so far
        char ch;
        // Read one character at a time; whitespace (spaces, tabs, newlines)
        // is skipped automatically by operator>> so any spacing style works.
        while (count < SIZE * SIZE && in >> ch) {
            int row = count / SIZE;
            int col = count % SIZE;
            if (ch >= '1' && ch <= '9') {
                original.cells[row][col] = ch - '0';   // filled cell
                count++;
            } else if (ch == '_' || ch == '0' || ch == '.') {
                original.cells[row][col] = EMPTY;      // empty cell
                count++;
            }
            // any other character (e.g., '|' or '-') is ignored
        }

        if (count != SIZE * SIZE) {
            return false;                    // file did not have 81 cells
        }

        working = original;                  // search works on a copy
        puzzleLoaded = true;
```

```
        return true;
    }

    // Runs the solver and collects all solutions.
    void solveAll() {
        solutions.clear();
        working = original;                    // always start from the original

        // If the given clues already conflict, there can be no solution.
        if (!initialPuzzleIsConsistent()) {
            return;
        }
        solve();                               // start the recursive search
    }

    // Prints the ORIGINAL puzzle (never modified by the solver).
    void printOriginal() const {
        printGrid(original);
    }

    // Prints every solution, or "No solution found".
    void printSolutions() const {
        if (solutions.empty()) {
            std::cout << "No solution found" << std::endl;
            return;
        }
        for (std::size_t i = 0; i < solutions.size(); i++) {
            std::cout << "Solution " << (i + 1) << " of "
                      << solutions.size() << ":" << std::endl;
            printGrid(solutions[i]);
        }
    }

    bool isLoaded() const { return puzzleLoaded; }

private:
    Grid original;                  // the puzzle exactly as read from the file
    Grid working;                   // scratch grid modified during the search
    std::vector<Grid> solutions;    // every complete solution found
    bool puzzleLoaded;              // true once a puzzle was read successfully

    // Sets every cell of a grid to EMPTY.
    static void clearGrid(Grid& g) {
        for (int r = 0; r < SIZE; r++)
            for (int c = 0; c < SIZE; c++)
                g.cells[r][c] = EMPTY;
    }

    // Checks whether placing 'num' at (row, col) breaks any Sudoku rule.
    // Looks at the row, the column, and the 3x3 block containing the cell.
    // (The cell itself is skipped so this also works on already-filled cells.)
```

```
    bool canPlace(int row, int col, int num) const {
        // Check the row and the column.
        for (int i = 0; i < SIZE; i++) {
            if (i != col && working.cells[row][i] == num) return false;
            if (i != row && working.cells[i][col] == num) return false;
        }
        // Check the 3x3 block: find its top-left corner first.
        int blockRow = (row / BLOCK) * BLOCK;
        int blockCol = (col / BLOCK) * BLOCK;
        for (int r = blockRow; r < blockRow + BLOCK; r++) {
            for (int c = blockCol; c < blockCol + BLOCK; c++) {
                if ((r != row || c != col) && working.cells[r][c] == num)
                    return false;
            }
        }
        return true;
    }

    // Verifies that the starting clues do not already violate any rule.
    bool initialPuzzleIsConsistent() const {
        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                int v = working.cells[r][c];
                if (v != EMPTY && !canPlace(r, c, v)) return false;
            }
        }
        return true;
    }

    // Finds the next empty cell (scanning left-to-right, top-to-bottom).
    // Returns true and sets row/col if one exists; false if the grid is full.
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < SIZE; row++)
            for (col = 0; col < SIZE; col++)
                if (working.cells[row][col] == EMPTY) return true;
        return false;
    }

    // ---------------------------------------------------------------
    // Recursive depth-first backtracking search.
    // Every call fills one empty cell. When no empty cells remain, the grid
    // is a complete valid solution and is saved. The search does NOT stop at
    // the first solution; it keeps backtracking to find all of them.
    // ---------------------------------------------------------------
    void solve() {
        int row, col;

        // Base case: no empty cell left -> record this solution.
        if (!findEmptyCell(row, col)) {
            solutions.push_back(working);
            return;
```

```
        }

        // Recursive case: try each digit 1-9 in the empty cell.
        for (int num = 1; num <= 9; num++) {
            if (canPlace(row, col, num)) {
                working.cells[row][col] = num;   // place the digit
                solve();                         // recurse on the next cell
                working.cells[row][col] = EMPTY; // backtrack: undo the move
            }
        }
        // If no digit worked, the loop ends and we return to the previous
        // call, which then tries its next candidate digit.
    }

    // Prints a grid with '_' for empty cells, with separators between blocks.
    static void printGrid(const Grid& g) {
        for (int r = 0; r < SIZE; r++) {
            if (r > 0 && r % BLOCK == 0) {
                std::cout << "------+-------+------" << std::endl;
            }
            for (int c = 0; c < SIZE; c++) {
                if (c > 0 && c % BLOCK == 0) std::cout << "| ";
                if (g.cells[r][c] == EMPTY) std::cout << "_";
                else                        std::cout << g.cells[r][c];
                if (c < SIZE - 1) std::cout << " ";
            }
            std::cout << std::endl;
        }
    }
};

// ------------------------------------------------------------------------
// Class: PuzzleManager
// Coordinates processing of the five puzzle files: loads each one, prints it,
// solves it, and prints the results.
// ------------------------------------------------------------------------
class PuzzleManager {
public:
    // Constructor: stores the names of the five puzzle files.
    PuzzleManager() {
        for (int i = 1; i <= NUM_PUZZLES; i++) {
            filenames.push_back("puzzle" + std::string(1, char('0' + i)) + ".txt");
        }
    }

    // Processes every puzzle file in order.
    void runAll() {
        for (std::size_t i = 0; i < filenames.size(); i++) {
            processPuzzle(filenames[i]);
        }
    }
```

```
private:
    static const int NUM_PUZZLES = 5;
    std::vector<std::string> filenames;

    // Loads, displays, solves, and reports on a single puzzle file.
    void processPuzzle(const std::string& filename) {
        std::cout << "==============================" << std::endl;
        std::cout << "Puzzle file: " << filename << std::endl;
        std::cout << "==============================" << std::endl;

        SudokuSolver solver;                        // one solver object per puzzle
        if (!solver.loadFromFile(filename)) {
            std::cout << "Error: could not read a valid puzzle from "
                      << filename << std::endl << std::endl;
            return;
        }

        std::cout << "Original puzzle:" << std::endl;
        solver.printOriginal();                     // show puzzle before solving
        std::cout << std::endl;

        solver.solveAll();                          // find ALL solutions
        solver.printSolutions();                    // print them (or no solution)
        std::cout << std::endl;
    }
};

// ---------------------------------------------------------------------------
// main: creates the manager object and runs all five puzzles.
// ---------------------------------------------------------------------------
int main() {
    PuzzleManager manager;
    manager.runAll();
    return 0;
}
```

1. Correctness

Gemini: Uses recursive backtracking, checks rows, columns, and 3×3 blocks, preserves the original puzzle, and prints every solution. However, it does not validate the starting clues or ensure the file contains exactly 81 valid cells, so malformed or already-invalid puzzles may be mishandled.

Claude: Uses the same solving approach but also checks whether the initial clues conflict and verifies that it reads 81 recognized cells. It still ignores unknown characters and does not reject extra values. Overall, Claude is more reliable, though its input validation could be stricter.

2. Time Complexity

Both programs have an approximate worst-case search time of O(9^E), where E is the number of empty cells. Row, column, and block checks take constant time for a fixed 9×9 board. Gemini advances directly to the next cell; Claude

rescans for an empty cell and stores solutions before printing. These are small overhead differences, not changes to the overall complexity. Choosing the most constrained empty cell could improve either solver.

3. Space Complexity

Both use fixed-size boards and a bounded recursive stack, so their working space is O(1) for a standard 9×9 puzzle. Gemini prints each solution immediately. Claude stores all S solutions, using O(S) additional space. Therefore, Gemini is more memory-efficient when a puzzle has many solutions.

4. Maintainability

Gemini is shorter and straightforward, but handles puzzle processing and solving in one class. Claude separates these responsibilities between SudokuSolver and PuzzleManager and uses focused methods for validation, searching, and solving. Its organization is easier to extend, although the solution vector adds unnecessary memory use and its input validation should be tightened. Overall, Claude is the stronger starting point.

F). Claude

I would choose Claude as the better starting point because it checks whether the original clues conflict and separates puzzle solving from file management. These features make its results more dependable and its structure easier to maintain. Both programs still have exponential worst-case search time, and Gemini has a memory advantage because it prints solutions immediately instead of storing them. Claude could be improved by reporting invalid or extra input characters and printing solutions as they are found. Overall, Claude offers the stronger balance of correctness and organization, while these changes would address its main limitations.

New Improved Code

```
/*
 * ============================================================================
 * Program Name  : EECS 348 Assignment 4 - Assignment4.cpp
 * Description   : Solves five 9x9 Sudoku puzzles with an object-oriented,
 *                 recursive depth-first search using backtracking. Checks the
 *                 puzzle format and starting clues, and prints EVERY solution.
 * Inputs        : puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt,
 *                 puzzle5.txt (digits 1-9 for clues; '_' for empty cells).
 * Outputs       : For each file, the filename, original puzzle, every valid
 *                 solution, or "No solution found". Invalid files show errors.
 * Author        : Conor Rothe
 * Creation Date : October 14, 2026
 * Collaborators : None
 * AI Sources    : Original Code 2 from Claude.
 *                 Revisions assisted with some ChatGPT.
 * Changes       : Strict input validation, bitmask checks, most-restricted-cell
 *                 selection, and printing solutions without storing them.
 * ============================================================================
 */

#include <cctype>
#include <fstream>
#include <iostream>
#include <string>
```

```cpp
// Source: Original AI-generated Code 2, revised with ChatGPT (OpenAI).
// SudokuSolver owns the puzzle state and performs the recursive search.
class SudokuSolver {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;
    static const int EMPTY = 0;

    // Keep the board inside a structure so the original can be copied easily.
    struct Grid {
        int cells[SIZE][SIZE];
    };

    // Initialize all cells and bookkeeping fields before loading a puzzle.
    SudokuSolver() : solutionCount(0), puzzleLoaded(false) {
        clearGrid(original);
        clearGrid(working);
        clearMasks();
    }

    // Read exactly 81 cells, ignoring whitespace but rejecting other characters.
    bool loadFromFile(const std::string& filename, std::string& error) {
        puzzleLoaded = false;
        solutionCount = 0;
        error.clear();
        clearGrid(original);
        clearGrid(working);
        clearMasks();

        std::ifstream input(filename.c_str());

        if (!input) {
            error = "could not open the file";
            return false;
        }

        int cellCount = 0;
        char ch;

        while (input.get(ch)) {
            // Spaces and newlines are allowed between Sudoku cells.
            if (std::isspace(static_cast<unsigned char>(ch))) {
                continue;
            }

            // Reject extra cells instead of ignoring trailing content.
            if (cellCount >= SIZE * SIZE) {
                error = "the file contains more than 81 puzzle cells";
                return false;
            }
```

```
            int value = EMPTY;

            if (ch >= '1' && ch <= '9') {
                value = ch - '0';
            }
            else if (ch != '_') {
                error = std::string("invalid character '") + ch +
                        "' (use digits 1-9 or '_')";
                return false;
            }

            // Store cells from top-left to bottom-right.
            original.cells[cellCount / SIZE][cellCount % SIZE] = value;
            ++cellCount;
        }

        // A puzzle must have all 81 positions.
        if (cellCount != SIZE * SIZE) {
            error = "the file contains fewer than 81 puzzle cells";
            return false;
        }

        if (input.bad()) {
            error = "an error occurred while reading the file";
            return false;
        }

        working = original;
        puzzleLoaded = true;

        return true;
    }

    // Print the original puzzle without modifying it.
    void printOriginal() const {
        printGrid(original);
    }

    // Solve the puzzle and print all solutions.
    void solveAll() {
        if (!puzzleLoaded) {
            std::cout << "No solution found\n";
            return;
        }

        working = original;
        solutionCount = 0;

        // Check whether the original numbers violate Sudoku rules.
        if (!initializeMasks()) {
            std::cout << "No solution found\n";
```

```
            return;
        }

        int emptyCount = 0;

        // Count how many cells still need to be filled.
        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {
                if (working.cells[row][col] == EMPTY) {
                    ++emptyCount;
                }
            }
        }

        // Begin recursive backtracking.
        solveRecursive(emptyCount);

        // Report when no valid solution exists.
        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        }
    }

private:
    static const int ALL_DIGITS = (1 << SIZE) - 1;

    Grid original;
    Grid working;

    // Bitmasks track numbers already used in each area.
    int rowUsed[SIZE];
    int colUsed[SIZE];
    int blockUsed[SIZE];

    // Count solutions without storing completed boards.
    unsigned long long solutionCount;
    bool puzzleLoaded;

    // Set every cell in a grid to empty.
    static void clearGrid(Grid& grid) {
        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {
                grid.cells[row][col] = EMPTY;
            }
        }
    }

    // Clear all row, column, and block bitmasks.
    void clearMasks() {
        for (int i = 0; i < SIZE; ++i) {
            rowUsed[i] = 0;
```

```
            colUsed[i] = 0;
            blockUsed[i] = 0;
        }
    }

    // Determine which 3x3 block contains a cell.
    static int blockIndex(int row, int col) {
        return (row / BLOCK) * BLOCK + col / BLOCK;
    }

    // Check the starting puzzle while building the bitmasks.
    bool initializeMasks() {
        clearMasks();

        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {

                int num = working.cells[row][col];

                if (num == EMPTY) {
                    continue;
                }

                // Convert the number into its corresponding bit.
                int bit = 1 << (num - 1);
                int block = blockIndex(row, col);

                // Check for duplicate numbers in row, column, or block.
                if ((rowUsed[row] & bit) ||
                    (colUsed[col] & bit) ||
                    (blockUsed[block] & bit)) {
                    return false;
                }

                // Mark the number as used.
                rowUsed[row] |= bit;
                colUsed[col] |= bit;
                blockUsed[block] |= bit;
            }
        }

        return true;
    }

    // Determine which numbers are allowed in an empty cell.
    int candidatesFor(int row, int col) const {
        int blocked = rowUsed[row] |
                      colUsed[col] |
                      blockUsed[blockIndex(row, col)];

        return ALL_DIGITS & ~blocked;
```

```
    }

    // Count how many available numbers are represented in a bitmask.
    static int countBits(int mask) {
        int count = 0;

        while (mask != 0) {
            mask &= mask - 1;
            ++count;
        }

        return count;
    }

    // Choose the empty cell with the fewest possible numbers.
    // This helps reduce unnecessary recursive searches.
    bool chooseMostRestrictedCell(int& bestRow, int& bestCol,
                                  int& bestMask) const {

        int lowestCount = SIZE + 1;

        bestRow = -1;
        bestCol = -1;
        bestMask = 0;

        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {

                // Skip cells that already contain numbers.
                if (working.cells[row][col] != EMPTY) {
                    continue;
                }

                // Find the possible numbers for this cell.
                int mask = candidatesFor(row, col);
                int count = countBits(mask);

                // Update the best cell if fewer numbers are possible.
                if (count < lowestCount) {
                    lowestCount = count;
                    bestRow = row;
                    bestCol = col;
                    bestMask = mask;

                    // Zero candidates means no solution on this branch.
                    // One candidate means this cell has a forced move.
                    if (count <= 1) {
                        return count == 1;
                    }
                }
            }
```

```
        }

        return bestRow != -1;
    }

    // Recursive backtracking algorithm.
    // Uses bitmasks and the most-restricted-cell strategy.
    void solveRecursive(int remaining) {

        // Base case: all empty cells have been filled.
        if (remaining == 0) {
            ++solutionCount;

            std::cout << "Solution " << solutionCount << ":\n";
```

G). Overall Improvement Summary

Correctness: I improved the input validation so the program checks for exactly 81 cells, rejects invalid characters, and detects extra or missing puzzle values. I also kept the validation that checks whether the original Sudoku puzzle contains conflicting numbers. These changes help prevent incorrect solutions and provide clearer error messages.

Execution Time: I improved the recursive backtracking algorithm by making it select the empty cell with the fewest possible numbers instead of always choosing the first empty cell. I also added bitmasks to track which numbers are already used in each row, column, and 3×3 block. This reduces unnecessary checking and can make difficult puzzles faster to solve. The worst-case time complexity remains O(9^E), where E is the number of empty cells.

Space Complexity: I removed the vector that stored every completed Sudoku solution. Instead, the program prints each solution immediately after finding it and uses a counter to track the total. This removes the O(S) solution-storage requirement, where S is the number of solutions. The program now only needs fixed-size boards, bitmasks, and memory for the recursive call stack.

Maintainability: I kept the original two-class structure, with SudokuSolver handling the puzzle and PuzzleManager handling the files. I added clearer comments, improved error messages, and separated important operations into methods for validation, candidate selection, and recursive solving. This makes the program easier to understand, debug, and modify later.